



Travlr Getaways
CS 465 Project Software Design Document
Version 1.0

Table of Contents

CS 465 Project Software Design Document	1
Table of Contents	2
Document Revision History	2
Instructions	2
Executive Summary	3
Design Constraints	3
System Architecture View	3
Component Diagram	3
Sequence Diagram	4
Class Diagram	4
API Endpoints	4
The User Interface	4

Document Revision History

Version	Date	Author	Comments
1.0	01/24/2026	Jared Harms	Module 3 completed

Instructions

Fill in all bracketed information on page one (the cover page), in the Document Revision History table, and below each header. Under each header, remove the bracketed prompt and write your own paragraph response covering the indicated information.

Executive Summary

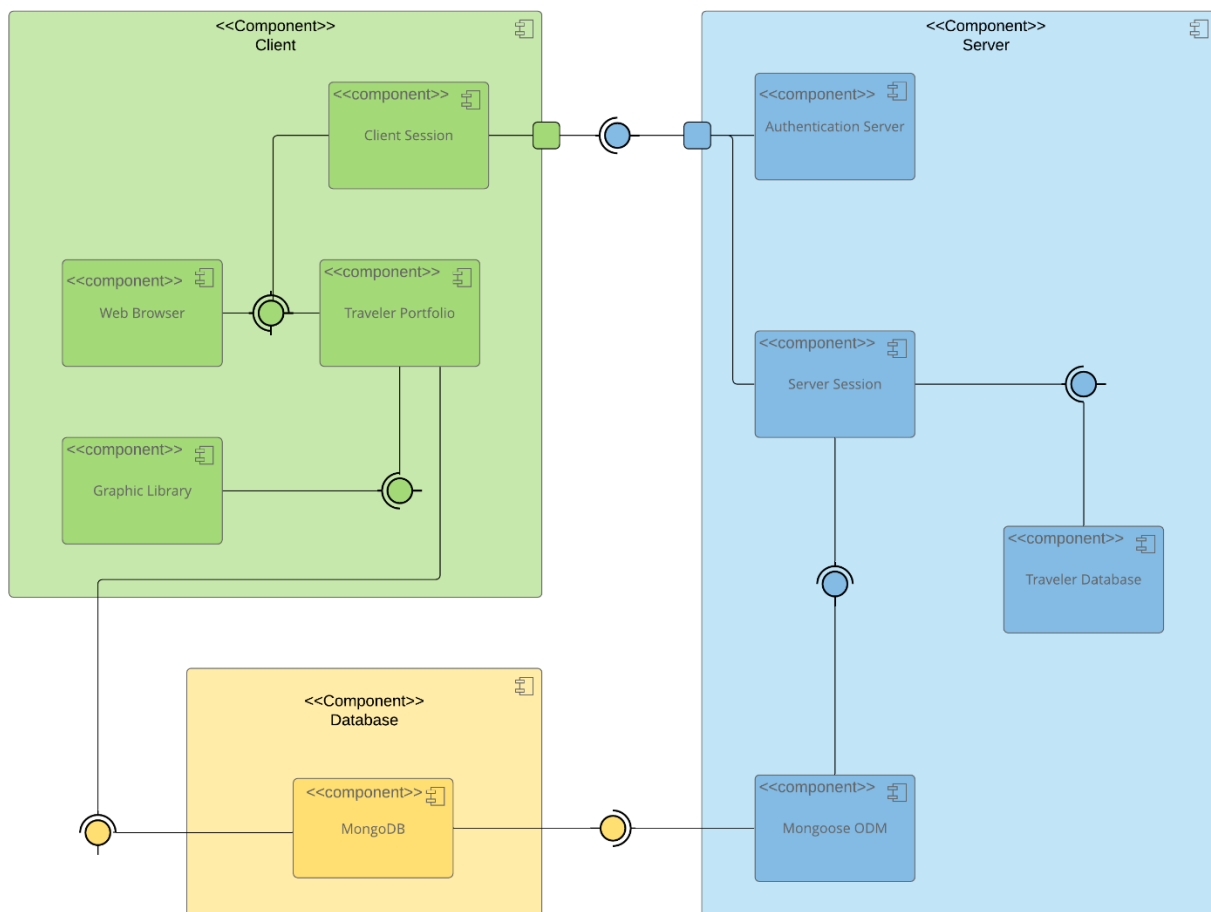
TravlR Getaways needs a scalable web application to enhance customer service and simplify administrative operations. The proposed solution uses the MEAN stack—MongoDB, Express, Angular, and Node.js—which is well suited for developing dynamic and responsive web applications. The system architecture is organized into three core components: a customer-facing website, a database, and an administrator single-page application (SPA).

Design Constraints

The customer-facing application is designed to deliver a seamless and intuitive user experience. Built with Angular, it provides a highly responsive and dynamically updated interface. This component allows travelers to efficiently browse offerings, complete bookings, and manage their travel plans, while integrating with the backend through proprietary APIs.

System Architecture View

Component Diagram



A text version of the component diagram is available: [CS 465 Full Stack Component Diagram Text Version](#).

The overall system architecture of the Travlr Getaways web application is represented through a component diagram that illustrates a three-tier Software Design Description (SDD). The client side is responsible for presenting the user interface, implemented using Angular. The backend consists of a Node.js server that handles application logic and communicates with a MongoDB database to securely and persistently store user data, including vacation package searches, bookings, and payment information. Data is retrieved and updated through HTTP requests between the client and server.

Built using Node.js and Express, the server side functions as a middle layer that handles client requests and communicates with the database. It is responsible for authenticating users, managing session information, and coordinating booking transactions. Express.js efficiently directs incoming requests to the appropriate handlers for data access and system operations.

Powered by MongoDB, the database serves as a robust system for efficiently storing and managing application data. It handles key information such as user profiles, authentication details, and travel preferences. In addition, it maintains comprehensive data on destinations, including geographic information, descriptive text, and related images. MongoDB also manages travel package data, covering pricing, itineraries, availability, and booking records. Its document-based structure provides flexible and scalable data storage and retrieval, making it well suited for the application's needs.

The core components of this project include a user interface built with frameworks such as React or Angular, a backend server powered by Node.js and the Express framework to handle data requests, and a flexible database solution like MongoDB. Additionally, Application Programming Interfaces (APIs) enable integration with external services, including payment gateways. Together, these components work seamlessly to deliver an efficient and user-friendly experience.

Sequence Diagram

<Illustrate the flow of logic in a web application by completing a sequence diagram. Insert an image of the sequence diagram here.>

<Describe the flow of logic in the web application based on the sequence diagram. Be sure to describe the interactions between the layers, or tiers, of the full stack application. It will be helpful to include significant processes such as Sign In, Trips, and Admin interactions when referring to the sequence diagram.>

Class Diagram

<Illustrate the JavaScript classes of the web application by completing a class diagram for the web application. Insert an image of the class diagram here.>

<Describe the JavaScript classes of the web application based on the class diagram.>

API Endpoints

<Exposing RESTful endpoints is a design approach to enable an application to participate in a larger ecosystem. Document each endpoint in the table below, including the HTTP method, purpose, URL, and notes.>

Method	Purpose	URL	Notes
GET	<Retrieve lis	</api/things>	<Returns all active things>
GET	<Retrieve single thing>	</api/things/:thingId>	<Returns single thing instance, identified by the thing ID passed on the request URL>

The User Interface

<Insert screenshots from the development of the SPA development to show the following: (1) a unique trip, added by you, (2) the Edit screen, and (3) the Update screen.>

<Summarize the Angular project structure and how it compares to the Express project structure. Be sure to describe the rich functionality provided by the SPA compared to a simple web application interaction. Describe the process of testing to make sure the SPA is working with the API to GET and PUT data in the database.>